ME 1600 – Python Project

Section 1: due on Dec. 7th 9:55 AM via Canvas
Section 4: due on Dec. 7th 2:15 PM via Canvas

Project rules

1. I would like you to demonstrate your knowledge level of the topics we covered in class and your proficiency in Python through a programming project. It should be a fully-fledged Python program, written ***entirely from scratch by you***, satisfying the requirements specified in the "Python PROJECT" document **(READ THE REQUIREMENTS CAREFULLY!)**.
2. You are not permitted to 'cut and paste' or otherwise directly copy blocks of code **from other teams or elsewhere or using any form of AI** – this constitutes academic fraud and you will receive a 0. You must also **read and sign the "Academic Dishonesty Statement"** document available on Canvas. **Your grade will not be finalized until the agreement is signed.**
3. For any py-file (script/module), **include the group name** as part of your header.
4. You should submit the completed assignment **via Canvas** and upload **all the required files**. After submitting the material (**deadline: see above**), your group must share the work done with the rest of the class during a **8-minute presentation**, also held the same day.
5. The points (30) are assigned, as noted in the "**Rubric**" file uploaded **on Canvas**. Note that your **code grade** (team-based) will be **multiplied by a peer coefficient**. The peer coefficient will be equal to your peer grade (the sum of the grades that your team members and yourself assigned to you) divided by the maximum peer grade received in your group.

General Description

1. It is recommended that you **develop an algorithm on paper** before you start coding.
2. You must write an **easily readable, well-designed code**. This includes healthy use of comments and other best practices (meaningful variable names, descriptive header blocks, use of user-made and built-in functions, comments, functions descriptions)
3. Use **user-made functions** to accomplish different program tasks (**mandatory**) and store them in a module (**mandatory**). Remember: code that is faster for the programmer to write is also faster for Python to execute.
4. The user must be able to run the program **many times** without closing it.
5. The program must **read input data from a file**. The file name is considered fixed, so use it directly in your code (do not prompt the user for the file name).
6. The program must **check if specific input is in the range expected** (e.g., positive/negative numbers or within a particular interval). If the values inserted are not in the allowed range, the program should **point out the mistake and request another value** until the user complies.
7. The program must perform **a series of calculations to retrieve the results**. The calculations will have a certain level of complexity.
8. The program **must plot** some of the variables as a function of others.
9. The program must **save the results on a file**.